

NAME:SANTHOSH G

REG NO:192521035

COURSE NAME: DESIGN AND ANALYSIS
OF ALGORITHMS

COURSE CODE:CSA 0611

SLOT:C

FACULTY NAME:DR.S.TAMIL SELVAN

CO4_AT3

1. Given jobs (profit, deadline): [(100,2),(19,1),(27,2),(25,1),(15,3)], solve Job Sequencing. Also load containers with weights [10,20,30,40] into capacity 60 using greedy. Compare selection strategies and outputs. Analyze optimality. Evaluate complexity.

CODE:

```
jobs = [(100, 2), (19, 1), (27, 2), (25, 1), (15, 3)]
```

```
weights = [10, 20, 30, 40]
```

```
capacity = 60
```

```
jobs.sort(key=lambda x: x[0], reverse=True)
```

```
slots = [False] * 3
```

```
selected = []
```

```
profit = 0
```

```
for p, d in jobs:
```

```
    for i in range(min(d, 3) - 1, -1, -1):
```

```
        if not slots[i]:
```

```
            slots[i] = True
```

```
            selected.append((p, d))
```

```
            profit += p
```

```
            break
```

```
print("JOB SEQUENCING")
```

```
print("Selected Jobs:", selected)
```

```
print("Maximum Profit:", profit)
```

```
weights.sort()
```

```
loaded = []
```

```
total = 0
```

```
for w in weights:
```

```

    if total + w <= capacity:
        loaded.append(w)
        total += w

print("\nCONTAINER LOADING")
print("Loaded Containers:", loaded)
print("Total Weight:", total)
print("Remaining Capacity:", capacity - total)

print("\nCOMPARISON")
print("Job Sequencing Strategy: Select highest profit jobs first.")
print("Container Loading Strategy: Select lightest containers first.")
print("Job Sequencing is optimal for maximizing profit.")
print("Container Loading is greedy for maximizing number of containers.")

print("\nCOMPLEXITY")
print("Job Sequencing: O(n log n)")
print("Container Loading: O(m log m)")

```

OUTPUT:

Output

```

Total Weight: 60
Remaining Capacity: 0

COMPARISON
Job Sequencing Strategy: Select highest profit jobs first.
Container Loading Strategy: Select lightest containers first.
Job Sequencing is optimal for maximizing profit.
Container Loading is greedy for maximizing number of containers.

COMPLEXITY
Job Sequencing: O(n log n)
Container Loading: O(m log m)

=== Code Execution Successful ===

```